Data Cleaning Checklist for Survey Research

Fillable PDF. Click a blue box to type; tick the small squares. Save with **File › Save** and attach the saved PDF to your assignment. Works best in the free Adobe Acrobat Reader; macOS Preview and web browsers also work. If an answer does not fit its box, continue it on an extra page. Source: `Step_11_Data_Preparation_Analysis_and_Reporting/data_cleaning_checklist.md` (rebuild with `tools/make_fillable_pdfs.py`).

This checklist operationalizes the data cleaning pipeline described in the Step 11 README. Apply exclusions **strictly in order** and log every decision with a reason, date, and the number of cases removed at each step. Preserve the raw data file unaltered.

Template Exclusion Log

Step	Criterion	N removed	N remaining	Notes	Date
0	Raw export				
1a	Non-consent				
1b	Failed eligibility screener				
1c	Metadata contradiction				
2a	Duplicate IP / fingerprint				
2b	Bot/reCAPTCHA flag				
2c	External link not engaged				
3a	Speeder (< Xs completion time)			threshold: XXs	
3b	Attention-check failure				
3c	Straightliner				
4	AI-generated text flag				
FINAL	Analysis sample				

R Code Examples

Step 1 — Eligibility

```
library(dplyr)

# 1a: Remove non-consenters
df <- df |> filter(consent == "Yes")

# 1b: Remove failed screeners (example: must be 18+)
df <- df |> filter(age >= 18)

# 1c: Remove geolocation mismatches (Qualtrics exports LocationCountry)
df <- df |> filter(LocationCountry == "US")
```

Step 2 — Bot and Fraud Detection

```
# 2a: Duplicate IP addresses
df <- df |>
  group_by(IPAddress) |>
  filter(row_number() == 1) |>   # keep first response per IP
  ungroup()

# 2b: Qualtrics bot detection score (Q_RecaptchaScore; 0–1, higher = more human)
df <- df |> filter(Q_RecaptchaScore > 0.5 | is.na(Q_RecaptchaScore))

# 2c: External link engagement (Qualtrics click-tracking embedded data)
df <- df |> filter(link_clicked == 1)
```

Step 3 — Low-Quality Responses

```
# 3a: Speeders — flag responses below pilot-derived threshold (e.g., 120 seconds)
speeder_threshold <- 120  # seconds; set from pilot median / 3 rule of thumb
df <- df |> filter(`Duration (in seconds)` >= speeder_threshold)

# 3b: Attention check failure
# Assumes attention_check column has a known correct answer (e.g., "Strongly Agree")
df <- df |> filter(attention_check == "Strongly Agree")

# 3c: Straightliners — flag respondents who gave identical responses to all Likert items
likert_cols <- df |> select(starts_with("Q"))
sd_per_row <- apply(likert_cols, 1, sd, na.rm = TRUE)
df <- df[sd_per_row > 0, ]
```

Step 4 — AI-Generated Text (Heuristic Flagging)

```
# Flag responses with very low character variance or suspicious length
library(stringr)
df <- df |>
  mutate(
    text_len = str_length(open_response),
    is_suspicious = text_len < 10 | text_len > 2000  # adjust thresholds
  )
# Review flagged responses manually before removing
suspicious <- df |> filter(is_suspicious)
```

Step 5 — Missing Data

```
library(mice)   # Multiple imputation
library(naniar) # Missing data visualization

# Visualize missing data pattern
naniar::vis_miss(df)

# Multiple imputation (5 datasets) for MCAR/MAR data
imputed <- mice(df, m = 5, method = "pmm", seed = 123)
df_complete <- complete(imputed, 1)  # use dataset 1, or pool across all 5
```

Python Code Examples

Steps 1–3 in Python

```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv("raw_data.csv")

# 1a: Remove non-consenters
df = df[df["consent"] == "Yes"]

# 1b: Age eligibility
df = df[df["age"] >= 18]

# 2a: Duplicate IPs — keep first
df = df.drop_duplicates(subset=["IPAddress"], keep="first")

# 3a: Speeders
speeder_threshold = 120  # seconds
df = df[df["Duration (in seconds)"] >= speeder_threshold]

# 3b: Attention check
df = df[df["attention_check"] == "Strongly Agree"]

# 3c: Straightliners
likert_cols = [c for c in df.columns if c.startswith("Q")]
df["response_sd"] = df[likert_cols].std(axis=1)
df = df[df["response_sd"] > 0]
```

Missing Data in Python

```
import missingno as msno
from sklearn.impute import SimpleImputer

# Visualize
msno.matrix(df)

# Simple imputation (mean for continuous, most_frequent for ordinal)
imputer = SimpleImputer(strategy="most_frequent")
df[likert_cols] = imputer.fit_transform(df[likert_cols])
```

References

• van Buuren, S., & Groothuis-Oudshoorn, K. (2011). mice: Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software*, 45(3). https://doi.org/10.18637/jss.v045.i03

• Vaske, J. J. (2019). *Survey Research and Analysis* (2nd ed.). Venture Publishing.

• Naito, R., Zhao, J., Naidoo, R., & Chan, K. M. (2023). Private and civic actions as distinct types of individual engagement for transforming the exotic pet trade. People and Nature, 5(5), 1526-1538.